

AutoDL 服务器 (Linux + GPU) 手搓教程： WeNet U2++ 安克语音识别

适合：在 AutoDL 租了带 GPU 的 Linux 服务器、已建好 Python 3.10 虚拟环境 的小白。
你将通过 SSH / JupyterLab 在服务器上一步步把整个项目搓出来，并用 GPU 训练。

核心理念不变：建一步 → 跑一步 → 看到  再继续。

⚠ Linux 和 Windows 命令不同：换行符是 `\n`（不是 PowerShell 的 `\r\n`），
路径用 `/`（不是 `\`）。本教程所有命令都是 Linux 版，直接复制即可。

开始前必读：AutoDL 的 5 个关键常识

1. 项目和数据放 `/root/autodl-tmp/`

- `/root/autodl-tmp/` 是数据盘：空间大、关机不丢、可扩容。
- `/root/`（系统盘）空间小，重置系统/换镜像会清空。
- 🐯 所以我们整个项目都建在 `/root/autodl-tmp/` 下。

2. 访问 GitHub / 微软TTS 要开"学术加速"

- AutoDL 在国内，直连 GitHub、微软语音服务会很慢甚至失败。
- 开加速：`source /etc/network_turbo`（当前终端有效）。
- 不用时关掉：`unset http_proxy https_proxy`（有时加速会影响访问国内站点，比如 pip 装国内镜像反而变慢，按需开关）。

3. GPU 让训练快很多 —— 先用 `nvidia-smi` 确认能看到显卡。

4. 服务器没有麦克风 —— `infer_demo.py --mic` 用不了，只能用 `--wav` 文件 方式识别。

5. 怎么在服务器上编辑/创建文件（三选一，推荐第 1 种）：

- **JupyterLab（最简单）**：AutoDL 控制台点"JupyterLab"进去，左侧文件树可新建文件夹、新建文件、双击编辑、拖拽上传。
- **VSCode Remote-SSH**：本地 VSCode 装 `Remote - SSH` 插件，用 AutoDL 给的 SSH 指令连上去，体验和本地一样。
- **nano（纯终端）**：`nano 文件名` 编辑，`Ctrl+O` 回车保存，`Ctrl+X` 退出。
- 💡 你也可以直接把电脑桌面 `anker-asr-wenet` 里我生成好的文件，用 JupyterLab 拖拽上传到服务器，省去敲代码。代码内容没变，本教程只讲在 Linux 上怎么跑、怎么验证。

第 1 部分：连上服务器、激活虚拟环境、建项目目录

1.1 打开终端

- 方式A：AutoDL 控制台 → [JupyterLab] → 右上角 `Terminal`（黑色图标）开一个终端。
- 方式B：用本地终端 SSH 登录（AutoDL 实例页有"登录指令"和密码，复制执行）。

1.2 激活你已经建好的 Python 3.10 虚拟环境

你说已建好 venv，根据当初的建法二选一激活：

- 如果是 **conda 环境**（最常见）：

```
conda activate 你的环境名      # 比如 conda activate py310
```

忘了名字就：`conda env list` 查看。

- 如果是 **venv 目录**（python -m venv 建的）：

```
source /root/autodl-tmp/你的venv目录/bin/activate
```

✅ **检查点：**

```
python --version      # 显示 Python 3.10.x
which python           # 路径指向你的虚拟环境，而不是 /usr/bin/python
```

1.3 在数据盘建项目目录

```
cd /root/autodl-tmp
mkdir -p anker-asr-wenet
cd anker-asr-wenet
# 一次性把子目录都建好
mkdir -p conf local tools model docs exp data
pwd      # 应显示 /root/autodl-tmp/anker-asr-wenet
```

✅ **检查点：** `ls` 能看到 `conf local tools model docs exp data` 这些文件夹。

之后所有命令都在 `/root/autodl-tmp/anker-asr-wenet` 这个目录下执行。

第 2 部分：确认 GPU + 装 PyTorch（GPU 版）

2.1 看显卡

```
nvidia-smi
```

✅ **检查点：** 打印出一个表格，能看到显卡型号（如 RTX 3090）、显存、CUDA Version。看到了就说明 GPU 正常。

2.2 装 GPU 版 PyTorch

AutoDL 有些镜像自带 torch，但你是自己新建的 venv，多半要自己装。

Linux 上 `pip install torch` 默认就是带 CUDA 的 GPU 版。

```
pip install torch torchaudio
```

如果你的实例 CUDA 较老、上面装的 torch 跑不起来，再用指定 CUDA 版本的方式（cu121 对应 CUDA 12.1）：

```
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cu121
```

✅ **检查点**（最重要的一步，务必看到 `True`）：

```
python -c "import torch; print('CUDA可用:', torch.cuda.is_available()); print('显卡:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else '无')"
```

打印 `CUDA可用: True` → GPU 版 PyTorch 装好了。

若是 `False`：说明装成了 CPU 版，用上面 `--index-url` 那条命令重装。

第 3 部分：装其余依赖 + ffmpeg

3.1 创建 `requirements.txt`

在项目根目录建 `requirements.txt`，内容用桌面标准答案（torch 那两行可留着，已装会自动跳过）。

JupyterLab：左侧文件树进到项目目录 → 右键 New File 改名 `requirements.txt` → 双击粘贴内容 → `Ctrl+S` 保存。

或 `nano requirements.txt` 粘贴后 `Ctrl+O` 回车 `Ctrl+X`。

3.2 装 Python 依赖

```
pip install -r requirements.txt
```

3.3 装 ffmpeg（Linux 上用 apt，AutoDL 是 root，直接装）

```
apt-get update && apt-get install -y ffmpeg
```

✅ **检查点**：

```
ffmpeg -version          # 打印版本信息
python -c "import torchaudio, edge_tts, soundfile; print('deps ok')"
```

两条都正常 → 依赖齐了。

💡 服务器没声卡，`import sounddevice` 可能告警，**不影响**（我们只用 `--wav` 不用麦克风）。

第 4 部分：跑教学版 U2++（先理解模型，不依赖 WeNet）

4.1 创建 `model/u2pp_conformer_min.py`

在 `model/` 下建该文件，内容用桌面标准答案。

4.2 运行

```
python model/u2pp_conformer_min.py
```

✅ **检查点：**打印"参数量 7.97M / 编码器输出 (2,24,144) / CTC损失 / L2R / R2L / 联合损失 / 结构验证通过 ✅"。这一步让你亲眼看到 Conformer + CTC + **双向解码器** 跑通了。

第 5 部分：生成语音数据集（要联网，开学术加速）

edge-tts 要访问微软语音服务器，国内直连可能失败，**先开学术加速**。

5.1 创建 `local/generate_anker_corpus.py`

在 `local/` 下建该文件，内容用桌面标准答案。指令清单 `ANKER_COMMANDS` 可自行增删。

5.2 开加速并生成

```
source /etc/network_turbo # 开学术加速
python local/generate_anker_corpus.py --out data/audio --repeat 3
```

GPU 服务器跑得快，`--repeat` 可以开大点（如 3~5）多造点数据，训练效果更好。

✅ **检查点：**

```
ls data/audio | head # 看到一堆 .wav
ls data/audio | wc -l # 数一下有多少个文件
cat data/audio/metadata.tsv | head # 看到 路径<Tab>文字
```

有几百个 wav + metadata.tsv → 成功。

- ❌ 报网络超时/连接失败 → 确认 `source /etc/network_turbo` 已执行；还不行就多重试几次。
- ❌ 报 ffmpeg 找不到 → 回第 3.3 装 ffmpeg。

第 6 部分：准备 WeNet 数据格式

6.1 创建 `local/prepare_data.py`

在 `local/` 下建该文件，内容用桌面标准答案。

6.2 运行

```
python local/prepare_data.py --audio_dir data/audio --out_dir data
```

✅ 检查点:

```
head -n 2 data/train/data.list      # 每行一个 JSON
cat data/dict/units.txt | head      # 字 编号
wc -l data/train/data.list data/dev/data.list data/test/data.list
```

三个 data.list + units.txt 都生成 → 成功。

第 7 部分：计算 CMVN

7.1 创建 tools/make_cmvn.py

在 tools/ 下建该文件，内容用桌面标准答案。

7.2 运行

```
python tools/make_cmvn.py --data_list data/train/data.list --out data/train/global_cmvn
```

✅ 检查点: 进度条跑完，打印"CMVN 完成"，生成 data/train/global_cmvn。

第 8 部分：安装 WeNet 训练代码（开学学术加速）

```
source /etc/network_turbo          # GitHub 加速
cd /root/autodl-tmp
git clone https://github.com/wenet-e2e/wenet.git
cd wenet
pip install -e .
cd /root/autodl-tmp/anker-asr-wenet # 切回项目目录!
```

✅ 检查点:

```
python -c "import wenet; print(wenet.__file__)" # 打印 wenet 安装路径
python -m wenet.bin.train --help | head         # 打印训练参数帮助
```

能打印帮助 → WeNet 训练代码就绪。

- ❌ git clone 太慢/失败 → 确认开了学术加速；或用 AutoDL 的代码加速地址（控制台有说明）。
- ❌ No module named wenet.bin → 没装成功，确认在 wenet 目录里执行了 pip install -e .，且虚拟环境是激活的。

第 9 部分：用 GPU 训练 U2++ Conformer (重点)

9.1 创建 `conf/train_u2pp_conformer.yaml`

在 `conf/` 下建该文件，内容用桌面标准答案。

GPU 显存够的话，可以把 `yaml` 里的 `batch_size` 调大（如 32），训练更快更稳。

9.2 指定用哪块 GPU

```
export CUDA_VISIBLE_DEVICES=0      # 用第 0 块卡（单卡就是它）
```

9.3 开始训练 —— 先试方案 A，报分布式错误就用方案 B

方案 A（多数版本可用，简单）：

```
python -m wenet.bin.train \
  --config conf/train_u2pp_conformer.yaml \
  --data_type raw \
  --symbol_table data/dict/units.txt \
  --train_data data/train/data.list \
  --cv_data data/dev/data.list \
  --model_dir exp/u2pp_conformer \
  --cmvn data/train/global_cmvn \
  --num_workers 4 \
  --pin_memory
```

方案 B（较新版本的 WeNet 要求用 `torchrun` 分布式启动）：

```
torchrun --standalone --nnodes=1 --nproc_per_node=1 \
  $(python -c "import
os,wenet;print(os.path.join(os.path.dirname(wenet.__file__),'bin','train.py'))") \
  --train_engine torch_ddp \
  --config conf/train_u2pp_conformer.yaml \
  --data_type raw \
  --symbol_table data/dict/units.txt \
  --train_data data/train/data.list \
  --cv_data data/dev/data.list \
  --model_dir exp/u2pp_conformer \
  --cmvn data/train/global_cmvn \
  --num_workers 4 \
  --pin_memory
```

✅ 检查点：

- 终端一轮轮打印 `Epoch 0 ... loss ...`，loss 总体下降。
- 另开一个终端实时看 GPU 占用（确认真的在用 GPU）：

```
watch -n 1 nvidia-smi
```

能看到你的 python 进程占着显存、GPU 利用率 >0 → GPU 训练成功。

- `exp/u2pp_conformer/` 下陆续出现 `0.pt 1.pt ...` 和 `train.yaml`，训完有 `final.pt`。

✗ 报 `unexpected key / unrecognized arguments` → 你的 WeNet 版本字段不同。

- 配置字段问题：按报错去 `conf/train_u2pp_conformer.yaml` 删/改对应行（核心 `conformer/bitransformer/ctc` 别动）。
- 命令参数问题：先跑 `python -m wenet.bin.train --help` 看你这版到底支持哪些参数，照着改。
- 还可参考你 clone 下来的 `wenet/examples/aisHELL/s0/` 里的 `run.sh`，那是和你版本完全匹配的范例。

✗ 报 CUDA out of memory（显存不够）→ 把 yaml 的 `batch_size` 调小（如 8）。

✗ 想后台挂着训练、断开 SSH 也不停 → 用 `nohup`：

```
nohup python -m wenet.bin.train ...(同上参数)... > train.log 2>&1 &
tail -f train.log           # 实时看日志，Ctrl+C 只是退出看日志、不杀训练
```

第 10 部分：解码评测（算 CER）

10.1 创建 `recognize.py`

在项目根目录建 `recognize.py`，内容用桌面标准答案。

默认它用 CPU 解码（测试集很小，够快）。想用 GPU 解码：把 `recognize.py` 里 `"--gpu", "-1"` 改成 `"--gpu", "0"` 即可（可选，不改也能跑）。

10.2 运行

```
python recognize.py \
  --config exp/u2pp_conformer/train.yaml \
  --checkpoint exp/u2pp_conformer/final.pt \
  --test_data data/test/data.list \
  --dict data/dict/units.txt \
  --mode attention_rescoring
```

没有 `final.pt` 就用序号最大的：先 `ls exp/u2pp_conformer/*.pt` 看一下，把 `--checkpoint` 换成它。

✓ **检查点**：逐条打印 ✓/✗ 参考：xxx | 识别：xxx，最后 CER（字错误率）= xx.xx%。

做对比实验（写报告用）：把 `--mode` 换成 `ctc_greedy_search` / `ctc_prefix_beam_search` / `attention` / `attention_rescoring` 各跑一次，记录 CER 填进 `docs/答辩大纲.md` 的表格。

第 11 部分：单条音频识别 Demo（服务器无麦克风，用 --wav）

11.1 创建 infer_demo.py

项目根目录建 infer_demo.py，内容用桌面标准答案。

11.2 运行（识别一个测试音频）

```
# 先取测试集第一条的 wav 路径
WAV=$(head -n1 data/test/data.list | python -c "import sys,json;print(json.loads(sys.stdin.read())['wav'])")
echo $WAV

python infer_demo.py \
  --config exp/u2pp_conformer/train.yaml \
  --checkpoint exp/u2pp_conformer/final.pt \
  --dict data/dict/units.txt \
  --wav "$WAV"
```

✅ **检查点：**打印  识别结果：【打开降噪】之类 → 全流程打通！

想识别自己的录音：在电脑上录一段说安克指令的 wav，用 JupyterLab 拖拽上传到服务器，把 --wav 换成它的路径即可。（脚本会自动转成 16k 单声道。）

第 12 部分：一键脚本（可选收尾）

创建 run.sh（内容用桌面标准答案），以后可分阶段重跑：

```
chmod +x run.sh
bash run.sh # 从头到尾
stage=3 stop_stage=3 bash run.sh # 只重跑训练那一步
stage=4 stop_stage=4 bash run.sh # 只重跑解码评测
```

注意：run.sh 里训练命令用的是方案 A，若你这版要方案 B（torchrun），把脚本里训练那段替换一下。

第 13 部分：AutoDL 省钱 & 保存成果（很重要）

1. 训练完及时关机，别让 GPU 空转烧钱。
2. 数据/模型务必在 /root/autodl-tmp/（关机不丢）；放 /root/ 的东西重置镜像会没。
3. 想保留环境又省钱：用 AutoDL 的「无卡模式开机」整理/下载文件（便宜很多，但没 GPU）。
4. 把训练好的模型下载到本地：JupyterLab 文件树里右键 exp/u2pp_conformer/final.pt → Download；或控制台的文件管理下载。
5. 重要文件也可传到网盘/对象存储备份。

全流程速查表（建文件 → 跑 → 验证）

步	文件	命令（关键）	看到什么
1	—	<code>conda activate / source ../activate</code>	python 3.10、which 指向 venv
2	—	<code>pip install torch torchaudio</code>	CUDA可用：True
3	requirements.txt	<code>pip install -r requirements.txt + apt install ffmpeg</code>	deps ok
4	model/u2pp_conformer_min.py	<code>python model/u2pp_conformer_min.py</code>	结构验证通过
5	local/generate_anker_corpus.py	<code>source /etc/network_turbo</code> + 运行	data/audio 一堆 wav
6	local/prepare_data.py	运行	data.list + units.txt
7	tools/make_cmvn.py	运行	global_cmvn
8	(装 WeNet)	<code>git clone + pip install -e .</code>	train --help 有输出
9	conf/train_u2pp_conformer.yaml	<code>CUDA_VISIBLE_DEVICES=0</code> + 训练	loss 下降、GPU 占用、出 *.pt
10	recognize.py	运行	CER = xx%
11	infer_demo.py	<code>--wav</code> 运行	识别结果

附录：Linux / AutoDL 报错急救表

报错关键词	原因	解决
CUDA可用：False	装成了 CPU 版 torch	用 <code>--index-url ../cu121</code> 重装 torch
git clone 卡住/失败	没开学术加速	<code>source /etc/network_turbo</code> 再 clone
edge-tts 网络超时	没开加速/微软不通	开学术加速；多重试
No module named wenet.bin	WeNet 没装好	在 wenet 目录 <code>pip install -e .</code> (venv 要激活)
ffmpeg: command not found	没装 ffmpeg	<code>apt-get install -y ffmpeg</code>
CUDA out of memory	显存不够	yaml 里 <code>batch_size</code> 调小
unrecognized arguments	WeNet 版本参数不同	<code>--help</code> 查参数；参考 <code>wenet/examples/aisHELL/s0/run.sh</code>

报错关键词	原因	解决
训练要 torchrun/DDP	新版需分布式启动	用第 9 部分「方案 B」
SSH 断开训练就停	前台进程随会话结束	用 <code>nohup ... &</code> 后台跑
磁盘空间不足	写到了系统盘	项目放 <code>/root/autodl-tmp/</code>
关机后文件没了	放在了 <code>/root/</code>	放 <code>/root/autodl-tmp/</code> ，并下载备份

🎉 在 GPU 服务器上手搓完成！原理见 `docs/原理讲解.md`，报告/答辩见 `docs/答辩大纲.md`。
GPU 训练比 CPU 快很多，记得**训练完关机省钱、成果存数据盘**。🎧